

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD

TEAM DETAILS

S.NO	NAME	COLLEGE NAME	CONTACT
1	HARINI A	St.Joseph's College for Women,Tirupur.	hariniangamuthu@gmail.com
2	RAMITHA S	St.Joseph's College for Women,Tirupur.	ramirami85711@gmail.com
3	AFREEN M	St.Joseph's College for Women,Tirupur.	afreenafreen7615@gmail.com
4	NANDHINI M	St.Joseph's College for Women,Tirupur.	Nandhini.murugan500@gmail.com

PROJECT DETAILS

Title:

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD

In today's digital world, organizations increasingly depend on cloud technologies and online applications to support their operations. As businesses adopt cloud platforms and remote working environments, the need for secure access to applications and data becomes more critical. Traditional security systems that rely on network boundaries are no longer effective in protecting modern IT infrastructures. Therefore, advanced security platforms and identity management systems are required to ensure secure access control.

Zscaler is a cloud-based security platform that provides secure internet access, threat protection, and Zero Trust Network Access (ZTNA). It helps organizations protect their network traffic and ensure that users can securely connect to applications regardless of their location.

Similarly, **Microsoft Azure Active Directory** is a cloud-based identity and access management service that allows organizations to manage user identities, authentication, and authorization in a centralized manner.

Integrating Zscaler with Azure Active Directory enables organizations to enforce identity-based security policies and provide secure authentication mechanisms such as Single Sign-On (SSO). This integration ensures that users are verified before accessing applications and network resources, thereby improving security and simplifying access management. The project focuses on understanding the integration process and analyzing how identity-based access control enhances security in modern cloud-based environments.

Abstract:

With the increasing use of cloud services and remote access, organizations face new challenges in securing user authentication and managing access to digital resources. Traditional security approaches are often insufficient to handle modern cloud-based environments. This project focuses on integrating **Zscaler**, a cloud security platform, with Identity Providers (IdPs) such as **Microsoft Azure Active Directory** to improve secure identity and access management. The integration enables organizations to authenticate users through a centralized identity system while maintaining secure network access. By combining the cloud security capabilities of Zscaler with the identity management features of Azure Active Directory, organizations can implement features such as Single Sign-On (SSO), identity-based access control, and enhanced authentication mechanisms. This approach improves overall security, simplifies user access management, and ensures that only authorized users can access organizational resources in a cloud environment.

Use Case Scenarios:

Scenario 1: User Authentication through Identity Provider

Organizations must ensure that only authorized users can access their applications and network resources. When a user attempts to log in to an application protected by Zscaler, the authentication request is redirected to the identity provider Microsoft Azure Active Directory. The user enters their credentials, and the identity provider verifies the username and password. After successful authentication, the identity information is shared with the Zscaler platform, which evaluates security policies before granting access to the requested resource. This process ensures secure and centralized identity verification.

Output:

- The user credentials are verified by the identity provider.
- Authentication is securely completed through Azure Active Directory.
- Zscaler validates the identity before allowing access.
- Authorized users gain secure access to the requested application.

Scenario 2: Single Sign-On (SSO) for Multiple Applications

In large organizations, employees often need to access multiple cloud applications for their daily tasks. Repeatedly entering credentials for each application can be inconvenient and may lead to security risks. By integrating Zscaler with Azure Active Directory, the system supports Single Sign-On (SSO). The user logs in once using their organization credentials, and the authentication token is used to access multiple applications without requiring repeated logins. This improves both security and user productivity.

Output:

- The user logs in once using their organizational credentials.
- Azure Active Directory authenticates the user identity.
- Zscaler recognizes the authenticated session.
- The user can securely access multiple applications without repeated login.

Scenario 3: Secure Remote Access for Employees

Employees working from home or other remote locations need secure access to company resources. When a remote employee attempts to access an application, Zscaler verifies the user's identity through Azure Active Directory before granting access. The system checks security policies and ensures that the request comes from an authorized user. If the authentication is successful and security conditions are met, the employee is granted secure access to the required application without exposing the entire network.

Output:

- The remote user attempts to access the organization's application.
- The system verifies the user identity through Azure Active Directory.
- Security policies are checked before allowing access.
- A secure connection is established for the authorized application.

Scenario 4: Role-Based Access Control

Different employees in an organization require different levels of access based on their roles and responsibilities. With identity-based access management, Azure Active Directory maintains user roles and permissions. When a user requests access to a resource, Zscaler checks the role information provided by Azure AD and applies the appropriate security policy. This ensures that users only access the resources necessary for their work and prevents unauthorized access to sensitive systems.

Output:

- The system identifies the user role through Azure Active Directory.
- Zscaler applies security policies based on the user role.
- Access is granted only to authorized resources.
- Sensitive applications remain protected from unauthorized users.

Scenario 5: Multi-Factor Authentication for Secure Access

For highly sensitive applications, organizations require additional security verification beyond passwords. In this scenario, when a user attempts to access a secure application, Azure Active Directory enforces Multi-Factor Authentication (MFA). The user must provide an additional verification method such as a one-time password or mobile authentication code. Once the verification is completed successfully, Zscaler allows the user to access the application securely.

Output:

- The user provides primary login credentials.
- Azure Active Directory requests additional authentication.
- The user completes Multi-Factor Authentication verification.
- Zscaler grants secure access to the requested application.

Scenario 6: Secure Access to Cloud Applications

Organizations use various cloud-based applications for communication, collaboration, and data management. When a user attempts to access a cloud application, Zscaler first checks whether the user is authenticated through Azure Active Directory. The platform then evaluates security policies such as user identity, location, and device information before allowing access. This ensures that cloud applications remain secure from unauthorized or suspicious access attempts.

Output:

- The user attempts to access a cloud application.
- The system verifies the user identity through Azure Active Directory.
- Zscaler checks security policies and user conditions.
- Secure access to the cloud application is granted.

Scenario 7: Monitoring and Logging User Access

Monitoring user activity is important for maintaining security and detecting potential threats. In this scenario, both Zscaler and Azure Active Directory record authentication attempts and access activities. Administrators can review these logs to monitor user behavior, identify unusual activities, and take necessary security actions. This logging mechanism helps organizations maintain compliance and improve their overall security posture.

Output:

- User login and access activities are recorded.
- Authentication logs are maintained by Azure Active Directory.
- Zscaler records network access and security events.
- Administrators can monitor and analyze user access patterns.

Scenario 8: Risk-Based Adaptive Access Control

When a user attempts to access an application, the system evaluates additional factors such as login location, device type, and user behavior. The authentication request is verified through Microsoft Azure Active Directory, which checks the user identity and determines the risk level of the login attempt. Based on this evaluation, Zscaler applies appropriate security policies before granting access. If the login appears suspicious, additional verification such as Multi-Factor Authentication is required.

Output:

- The system evaluates login location and device details.
- Azure Active Directory verifies the user identity.
- Zscaler applies security policies based on risk level.
- Additional verification is required for suspicious access attempts.

STAGE 1

Title of the project:

Integrate Zscaler with Identity Providers (IdPs) such as Azure AD

Overview:

In the modern digital environment, organizations rely heavily on cloud computing, remote access, and online applications to support their business operations. With employees accessing organizational resources from different locations and devices, ensuring secure authentication and controlled access has become a major challenge. Traditional network security models, which rely on perimeter-based protection, are no longer sufficient to secure modern IT infrastructures.

As a result, organizations are increasingly adopting cloud-based security platforms and identity management systems to protect their resources. One of the most effective approaches to improving security is integrating advanced cloud security solutions with identity providers. Zscaler is a cloud-based security platform that provides secure internet access, threat protection, and Zero Trust Network Access (ZTNA). It acts as a security gateway between users and applications, ensuring that all traffic is inspected and protected from potential cyber threats. By implementing a Zero Trust model, Zscaler ensures that users are authenticated and verified before they are allowed to access organizational resources.

At the same time, identity management plays a critical role in controlling access to applications and systems. Microsoft Azure Active Directory is a widely used cloud-based identity and access management service that allows organizations to manage user identities, authentication processes, and access permissions in a centralized environment. Azure Active Directory supports advanced authentication methods such as Single Sign-On (SSO), Multi-Factor Authentication (MFA), and conditional access policies, which help organizations strengthen their security posture.

The integration of Zscaler with Azure Active Directory enables organizations to combine network security with identity-based access control. When a user attempts to access an

application or service, the authentication request is first verified by Azure Active Directory. After the user's identity is confirmed, Zscaler evaluates security policies and ensures that the connection meets organizational security standards before granting access. This process helps prevent unauthorized access and ensures that only legitimate users can access critical resources.

Another important advantage of this integration is the implementation of Single Sign-On (SSO). With SSO, users can log in once using their organizational credentials and gain access to multiple applications without repeatedly entering their login details. This improves user experience while maintaining a high level of security. In addition, Multi-Factor Authentication adds an extra layer of protection by requiring users to verify their identity through additional authentication methods.

The integration also supports secure remote access for employees working outside the organizational network. Instead of providing full network access through traditional VPN solutions, Zscaler ensures that users can access only the specific applications they are authorized to use. This reduces the risk of unauthorized access and improves overall network security.

Furthermore, the integration provides detailed monitoring and logging capabilities. Both Zscaler and Azure Active Directory maintain records of user authentication attempts and access activities. These logs help administrators monitor user behavior, detect suspicious activities, and maintain compliance with security policies.

Overall, integrating Zscaler with identity providers such as Azure Active Directory provides a modern and secure approach to managing user access in cloud-based environments. By combining identity management with advanced network security, organizations can protect sensitive data, improve access control, and ensure a secure digital infrastructure. This project focuses on understanding the integration process and demonstrating how identity-based security solutions can enhance the protection of organizational resources.

List of Vulnerability Table:

S.no	Category Name	Vulnerability Name	CWE/CVE - No
01	Broken Access Control	Unverified Ownership	CWE-283
02	Security Misconfiguration	Active Debug Code	CWE-489
03	Software Supply Chain Failures	Dependency on Vulnerable Third-Party Component	CWE-1395
04	Cryptographic Failures	Use of Weak Hash	CWE-328
05	Injection	Improper Neutralization of Equivalent Special Elements	CWE-76
06	Insecure Design	Incorrect User Management	CWE-286
07	Authentication Failures	Use of Hard-coded Password	CWE-259
08	Software or data integrity Failures	Untrusted Search Path	CWE-426
09	Security Logging and Alerting Failures	Insufficient Logging	CWE-778
10	Mishandling Exceptional Conditions	Divide By Zero	CWE-369

REPORTS

1)Vulnerability Name: Unverified Ownership

CWE/CVE: CWE-283

OWASP Category: Broken Access Control

Description: Unverified Ownership is a vulnerability that occurs when an application does not properly verify whether a user actually owns or has permission to access a particular resource. Attackers can manipulate identifiers such as user IDs, object IDs, or resource IDs to access or modify resources that belong to other users.

Business Impact: Attackers may gain unauthorized access to sensitive data or perform actions on resources owned by other users. This can result in data manipulation, unauthorized transactions, or exposure of confidential information. Such incidents may lead to data breaches, loss of customer trust, financial losses, and legal or regulatory consequences for the organization.

CWE-283: Unverified Ownership

Weakness ID: 283
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information:

Conceptual Operational Mapping Friendly Complete Custom

Description

The product does not properly verify that a critical resource is owned by the proper entity.

Common Consequences

Impact	Details
Gain Privileges or Assume Identity	Scope: Access Control An attacker could gain unauthorized access to system resources.

Potential Mitigations

Phase(s)	Mitigation
Architecture and Design; Operation	Very carefully manage the setting, management, and handling of privileges. Explicitly manage trust zones in the software.
Architecture and Design	Strategy: Separation of Privilege

Recommendation:

1. Implement Proper Authorization Checks:

Ensure that the system verifies whether the authenticated user is the legitimate owner of the resource before granting access.

2. Use Role-Based Access Control (RBAC):

Define user roles and permissions clearly so that users can access only the resources that they are authorized to use.

3. Avoid Direct Object References:

Do not expose direct identifiers such as sequential user IDs or resource IDs without proper validation and access control checks

4. Enforce Server-Side Validation:

Perform ownership verification on the server side instead of relying only on client-side checks.

5. Conduct Regular Security Testing:

Perform vulnerability scanning, penetration testing, and code reviews to identify and fix access control issues before deployment.

2)Vulnerability Name: Active Debug Code

CWE/CVE: CWE-489

OWASP Category: Security Misconfiguration

Description: Active Debug Code occurs when debugging features or test code remain enabled in a production environment. These debug messages may reveal internal system information such as file paths or configuration details.

Business Impact: Attackers may use exposed system information to identify other vulnerabilities. This can lead to unauthorized access, data exposure, and potential system compromise.

Debug code can be used to bypass authentication. For example, suppose an application has a login script that receives a username and a password. Assume also that a third, optional, parameter, called "debug", is interpreted by the script as requesting a switch to debug mode, and that when this parameter is given the username and password are not checked. In such a case, it is very simple to bypass the authentication process if the special behavior of the application regarding the debug parameter is known. In a case where the form is:

Example Language: **HTML**

(bad code)

```
<FORM ACTION="/authenticate_login.cgi">
  <INPUT TYPE=TEXT name=username>
  <INPUT TYPE=PASSWORD name=password>
  <INPUT TYPE=SUBMIT>
</FORM>
```

Then a conforming link will look like:

(informative)

```
http://TARGET/authenticate_login.cgi?username=...&password=...
```

An attacker can change this to:

(attack code)

```
http://TARGET/authenticate_login.cgi?username=&password=&debug=1
```

Which will grant the attacker access to the site, bypassing the authentication process.

Recommendation:

1. Remove Debug Code Before Deployment:

Ensure that all debugging statements and test code are removed before releasing the application.

2. Disable Debugging in Production:

Configure the production environment to disable debugging features.

3. Follow Secure Coding Practices:

Developers should ensure that development-only code is not included in production builds.

4. Perform Security Reviews:

Conduct code reviews and testing to detect leftover debug features.

5. Regular Security Testing:

Use vulnerability scanning and penetration testing to identify misconfigurations.

3)Vulnerability Name: Dependency on Vulnerable Third-Party Component

CWE/CVE: CWE-1395

OWASP Category: Software Supply Chain Failures

Description: This vulnerability occurs when an application uses third-party libraries or components that contain known security vulnerabilities. Outdated or insecure dependencies may expose the application to attacks.

Business Impact: Attackers can exploit vulnerable components to gain unauthorized access or execute malicious code. This may lead to data breaches or system compromise.

log4j, a Java-based logging framework, is used in a large number of products, with estimates in the range of 3 billion affected devices [\[REF-1317\]](#). When the "log4shell" (CVE-2021-44228) vulnerability was initially announced, it was actively exploited for remote code execution, requiring urgent mitigation in many organizations. However, it was unclear how many products were affected, as Log4j would sometimes be part of a long sequence of transitive dependencies. [\[REF-1316\]](#)

Recommendation:

1. Regularly Update Dependencies:

Keep all third-party libraries and components updated to the latest secure versions.

2. Monitor Vulnerability Databases:

Check for known vulnerabilities in dependencies using security databases.

3. Use Trusted Sources:

Download libraries only from official and trusted repositories.

4. Implement Dependency Management Tools:

Use tools that scan and detect vulnerable components.

5. Conduct Security Audits:

Perform regular audits of third-party dependencies.

4)Vulnerability Name: Use of Weak Hash

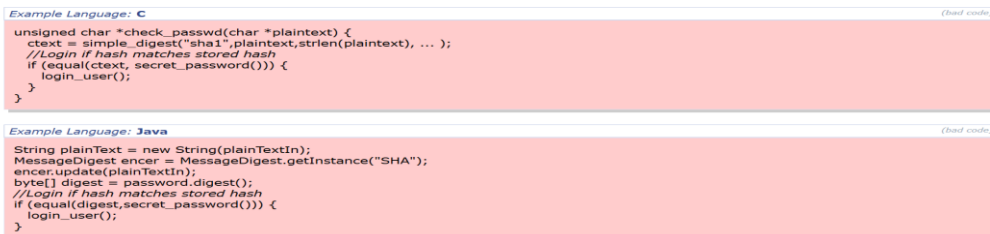
CWE/CVE: CWE-328

OWASP Category: Cryptographic Failures

Description: The use of weak hashing algorithms occurs when applications use outdated or insecure cryptographic hash functions such as MD5 or SHA-1. These algorithms are vulnerable to attacks such as collision and brute-force attacks.

Business Impact: Attackers may exploit weak hashes to crack stored passwords or sensitive information. This can lead to unauthorized account access, data breaches, and compromise of confidential information.

In both of these examples, a user is logged in if their given password matches a stored password:



The image shows two code snippets. The first is for C, titled 'Example Language: C', and the second is for Java, titled 'Example Language: Java'. Both snippets show a login function that calculates a hash of the input password and compares it to a stored hash. The C code uses 'simple_digest' with 'sha1' as the algorithm. The Java code uses 'MessageDigest.getInstance("SHA")'. Both snippets are highlighted in a light red box.

```
Example Language: C
unsigned char *check_passwd(char *plaintext) {
    ctext = simple_digest("sha1", plaintext, strlen(plaintext), ... );
    //Login if hash matches stored hash
    if (equal(ctext, secret_password())) {
        login_user();
    }
}

Example Language: Java
String plainText = new String(plainTextIn);
MessageDigest encr = MessageDigest.getInstance("SHA");
encr.update(plainTextIn);
byte[] digest = password.digest();
//Login if hash matches stored hash
if (equal(digest, secret_password())) {
    login_user();
}
```

This code relies exclusively on a password mechanism ([CWE-309](#)) using only one factor of authentication ([CWE-308](#)). If an attacker can steal or guess a user's password, they are given full access to their account. Note this code also uses SHA-1, which is a weak hash ([CWE-328](#)). It also does not use a salt ([CWE-759](#)).

Recommendation:

1. Use Strong Hashing Algorithms:

Use secure hashing algorithms such as SHA-256 or SHA-3.

2. Apply Salting Techniques:

Add random values (salt) to passwords before hashing.

3. Use Password Hashing Functions:

Use secure password hashing algorithms like bcrypt, PBKDF2, or Argon2.

4. Avoid Deprecated Algorithms:

Do not use outdated hashing methods such as MD5 or SHA-1.

5. Regular Security Updates:

Update cryptographic implementations regularly.

5)Vulnerability Name: Improper Neutralization of Equivalent Special Elements

CWE/CVE: CWE-76

OWASP Category: Injection

Description: This vulnerability occurs when an application fails to properly sanitize or validate user input containing special characters. Attackers may inject malicious commands or scripts into the application to manipulate system operations.

Business Impact: Attackers may exploit this vulnerability to execute unauthorized commands, access sensitive data, or compromise the system. This could result in data leakage, system disruption, or unauthorized system control.

▼ Relevant to the view "Research Concepts" (View-1000)			
Nature	Type	ID	Name
ChildOf	🟢	75	Failure to Sanitize Special Elements into a Different Plane (Special Element Injection)
▼ Relevant to the view "Software Development" (View-699)			
Nature	Type	ID	Name
MemberOf	🔴	137	Data Neutralization Issues
▼ Relevant to the view "Architectural Concepts" (View-1008)			
Nature	Type	ID	Name
MemberOf	🔴	1019	Validate Inputs

Recommendation:

1. Input Validation:

Validate all user inputs before processing them.

2. Use Parameterized Queries:

Avoid dynamic queries and use prepared statements.

3. Escape Special Characters:

Properly encode special characters to prevent injection attacks.

4. Implement Security Frameworks:

Use frameworks that automatically handle input sanitization.

5. Conduct Security Testing:

Regularly test the application for injection vulnerabilities.

6)Vulnerability Name: Incorrect User Management

CWE/CVE: CWE-286

OWASP Category: Insecure Design

Description: Incorrect User Management occurs when applications fail to properly handle user accounts, roles, or permissions. Weak user management processes may allow unauthorized users to gain access to restricted features.

Business Impact: Unauthorized users may gain access to sensitive resources or perform privileged actions. This can result in data manipulation, system misuse, and compromise of organizational security.

▼ Vulnerability Mapping Notes	
Usage	ALLOWED-WITH-REVIEW <i>(this CWE ID could be used to map to real-world vulnerabilities in limited situations requiring careful review)</i>
Reason	Abstraction
Rationale	This CWE entry is a Class and might have Base-level children that would be more appropriate
Comments	Examine children of this entry to see if there is a better fit

Recommendation:

1. Implement Role-Based Access Control:

Assign permissions based on user roles.

2. Enforce Strong Authentication:

Use secure login mechanisms.

3. Regularly Review User Accounts:

Remove inactive or unnecessary accounts.

4. Apply Least Privilege Principle:

Grant only the minimum permissions required.

5. Conduct Security Audits:

Review user management policies regularly.

7)Vulnerability Name: Use of Hard-coded Password

CWE/CVE: CWE-259

OWASP Category: Authentication Failures

Description: This vulnerability occurs when passwords are embedded directly in the application source code. Hard-coded passwords can be easily discovered if the code is exposed or reverse-engineered.

Business Impact: Attackers may obtain the hard-coded credentials and gain unauthorized access to systems or databases. This can lead to data breaches, system compromise, and unauthorized control of services.

The product contains a hard-coded password, which it uses for its own inbound authentication or for outbound communication to external components.



Recommendation:

1. Avoid Hard-coded Credentials:

Never store passwords directly in source code.

2. Use Secure Credential Storage:

Store passwords in secure configuration files or secret management systems.

3. Implement Environment Variables:

Use environment variables to store sensitive information.

4. Apply Encryption Techniques:

Encrypt sensitive credentials.

5. Perform Code Reviews:

Regularly review code to detect hard-coded secrets.

8)Vulnerability Name: Untrusted Search Path

CWE/CVE: CWE-426

OWASP Category: Software or data integrity Failures

Description: Untrusted Search Path occurs when an application loads executable files or libraries from directories that may be controlled by an attacker. This may allow attackers to place malicious files in those directories.

Business Impact: Attackers may execute malicious code by tricking the application into loading compromised files. This can result in system compromise, data theft, or unauthorized system control.

This program is intended to execute a command that lists the contents of a restricted directory, then performs other actions. Assume that it runs with setuid privileges in order to bypass the permissions check by the operating system.

```
Example Language: C (bad code)

#define DIR "/restricted/directory"

char cmd[500];
sprintf(cmd, "ls -l %480s", DIR);
/* Raise privileges to those needed for accessing DIR. */

RaisePrivileges(...);
system(cmd);
DropPrivileges(...);
...
```

This code may look harmless at first, since both the directory and the command are set to fixed values that the attacker can't control. The attacker can only see the contents for DIR, which is the intended program behavior. Finally, the programmer is also careful to limit the code that executes with raised privileges.

However, because the program does not modify the PATH environment variable, the following attack would work:

- ```
(attack code)
```
- The user sets the PATH to reference a directory under the attacker's control, such as "/my/dir/".
  - The attacker creates a malicious program called "ls", and puts that program in /my/dir
  - The user executes the program.
  - When system() is executed, the shell consults the PATH to find the ls program

## Recommendation:

### 1. Use Trusted Directories:

Load executable files only from secure and trusted locations.

### 2. Avoid Relative Paths:

Use absolute paths when referencing files.

### 3. Restrict File Permissions:

Limit write access to critical directories.

### 4. Implement Integrity Checks:

Verify the integrity of files before execution.

### 5. Regular Security Monitoring:

Monitor system directories for suspicious files.

## 9)Vulnerability Name: Insufficient Logging

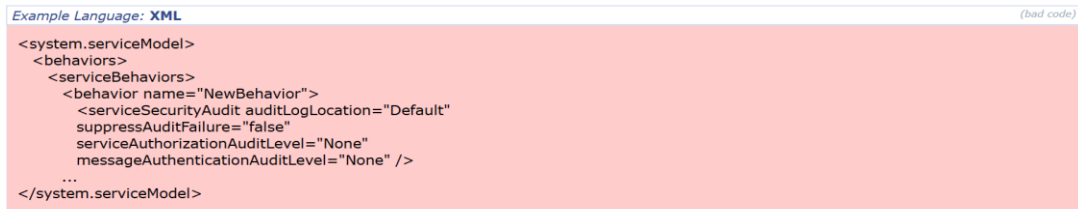
**CWE/CVE:** CWE-778

**OWASP Category:** Security Logging and Alerting Failures

**Description:** Insufficient Logging occurs when an application fails to record important security events such as login attempts, system errors, or suspicious activities. Without proper logging, it becomes difficult to detect and investigate security incidents.

**Business Impact:** Security breaches may go unnoticed due to lack of monitoring. Organizations may be unable to detect attacks or respond quickly to threats. This can lead to prolonged system compromise, data loss, and regulatory compliance issues.

The example below shows a configuration for the service security audit feature in the Windows Communication Foundation (WCF).

A screenshot of a code editor window showing an XML configuration snippet. The window has a title bar that says "Example Language: XML" and a small "(bad code)" icon in the top right corner. The XML code is as follows:

```
<system.serviceModel>
 <behaviors>
 <serviceBehaviors>
 <behavior name="NewBehavior">
 <serviceSecurityAudit auditLogLocation="Default"
 suppressAuditFailure="false"
 serviceAuthorizationAuditLevel="None"
 messageAuthenticationAuditLevel="None" />
 </behavior>
 </serviceBehaviors>
 </behaviors>
</system.serviceModel>
```

The previous configuration file has effectively disabled the recording of security-critical events, which would force the administrator to look to other sources during debug or recovery efforts.

Logging failed authentication attempts can warn administrators of potential brute force attacks. Similarly, logging successful authentication events can provide a useful audit trail when a legitimate account is compromised. The following configuration shows appropriate settings, assuming that the site does not have excessive traffic, which could fill the logs if there are a large number of success or failure events ([CWE-779](#)).

### Recommendation:

#### 1. Implement Comprehensive Logging:

Log all important security-related events.

#### 2. Monitor Logs Regularly:

Analyze logs to detect suspicious activities.

#### 3. Enable Alert Mechanisms:

Set up alerts for unusual system behavior.

#### 4. Secure Log Storage:

Protect logs from unauthorized modification.

#### 5. Conduct Regular Audits:

Review logs to ensure proper security monitoring.

## 10)Vulnerability Name: Divide By Zero

**CWE/CVE:** CWE-369

**OWASP Category:** Mishandling Exceptional Conditions

**Description:** Divide By Zero occurs when a program attempts to divide a number by zero without proper error handling. This can cause application crashes or unexpected behavior if the input is not properly validated.

**Business Impact:** Attackers may exploit this condition to cause system errors or application crashes, leading to service disruption. Frequent failures may affect system availability and user experience.

The following Java example contains a function to compute an average but does not validate that the input value used as the denominator is not zero. This will create an exception for attempting to divide by zero. If this error is not handled by Java exception handling, unexpected results can occur.

```
Example Language: Java (bad code)
public int computeAverageResponseTime (int totalTime, int numRequests) {
 return totalTime / numRequests;
}
```

By validating the input value used as the denominator the following code will ensure that a divide by zero error will not cause unexpected results. The following Java code example will validate the input value, output an error message, and throw an exception.

```
Example Language: Java (good code)
public int computeAverageResponseTime (int totalTime, int numRequests) throws ArithmeticException {
 if (numRequests == 0) {
 System.out.println("Division by zero attempted!");
 throw ArithmeticException;
 }
 return totalTime / numRequests;
}
```

### Recommendation:

#### 1. Validate Input Values:

Check input values before performing calculations.

#### 2. Implement Error Handling:

Use exception handling to manage invalid operations.

#### 3. Use Safe Programming Practices:

Ensure calculations handle edge cases properly.

#### 4. Perform Code Reviews:

Check code for potential logical errors.

#### 5. Conduct Thorough Testing:

Test applications with various input scenarios to prevent runtime errors

## STAGE – 2

### Overview:

Nessus is a widely used vulnerability assessment tool developed by Tenable, Inc.. It is designed to detect security weaknesses in systems, networks, and applications. Nessus operates as a remote vulnerability scanner, which means it can analyze target systems without requiring installation on those systems.

The tool works by sending different types of requests and probes to the target machine. These probes help identify open ports, running services, operating systems, and possible security flaws. The scanning process may involve network packets, service queries, and application-level requests to determine whether known vulnerabilities exist.

Nessus uses a plugin-based framework to perform vulnerability detection. Each plugin is responsible for identifying a specific security issue such as outdated software, configuration errors, or known exploits. Thousands of plugins are available, allowing Nessus to detect a wide range of vulnerabilities across operating systems, databases, web servers, and network devices.

During a scan, Nessus first performs host discovery to identify active systems within the network. After discovering the target, it gathers information about the operating system and running services. Based on this information, Nessus executes relevant plugins to test for potential vulnerabilities. When a weakness is identified, the tool records it and includes detailed information in the final report.

Once the scanning process is completed, Nessus generates a comprehensive report that lists all detected vulnerabilities along with their severity levels. The report also provides technical details, possible impacts, and recommended remediation steps to help security teams fix the issues.

Nessus can be used to scan individual systems, entire networks, and web applications. It also allows users to focus scans on specific ports or services if needed. Scans can be executed manually or scheduled at regular intervals to maintain continuous security monitoring.

### Basic Working Process of Nessus

1. Nessus establishes communication with the target system using protocols such as TCP, UDP, or ICMP.

2. It performs host discovery to identify active devices and determine their operating systems and services.
3. Relevant plugins are executed to test the target for known vulnerabilities.
4. A detailed report is generated showing vulnerability severity, technical description, and remediation guidance.

### **Advantages of Using Nessus**

- Extensive vulnerability detection: Nessus supports thousands of plugins that detect a wide variety of security issues.
- User-friendly interface: The tool provides a simple and intuitive interface for configuring and launching scans.
- Detailed vulnerability reports: Scan results include severity ratings, vulnerability descriptions, and recommended fixes.
- Scalable scanning capability: It can scan individual hosts, large networks, and web applications.
- Flexible scanning options: Users can run scans on demand or schedule them periodically.

Overall, Nessus is an effective vulnerability assessment tool that helps organizations identify security risks and strengthen the protection of their systems and networks.

• **Target website:** [zero.webappsecurity.com](https://zero.webappsecurity.com)

• **Target ip address:** [54.82.22.214](#)



**List of vulnerability:**

| <b>s.no</b> | <b>Vulnerability name</b>                                               | <b>Severity</b> | <b>plugins</b> |
|-------------|-------------------------------------------------------------------------|-----------------|----------------|
| 1           | SSL / TLS Renegotiation<br>Handshakes MiTM<br>Plaintext Data Injection  | Medium          | 42880          |
| 2           | SSL / TLS Versions<br>Supported                                         | Info            | 56984          |
| 3           | SSL/TLS Recommended<br>Cipher Suites                                    | Info            | 156899         |
| 4           | SSL/TLS Diffie-Hellman<br>Modulus <= 1024 Bits<br>(Logjam)              | Low             | 83875          |
| 5           | SSL Version 2 and 3<br>Protocol Detection                               | Critical        | 20007          |
| 6           | Nessus SYN scanner                                                      | Info            | 11219          |
| 7           | JQuery 1.2 < 3.5.0<br>Multiple XSS                                      | Medium          | 136929         |
| 8           | Integrate Zscaler with<br>Identity Providers (IdPs)<br>such as Azure AD | Info            | 19506          |
| 9           | Shor's Harvest Now<br>Decrypt Later                                     | Info            | 298387         |
| 10          | Patch Report                                                            | Info            | 66334          |

## REPORTS

### 1)Vulnerability Name: SSL / TLS Renegotiation Handshakes MiTM Plaintext Data

Injection

**Severity:** Medium

**Plugin:** 42880

**Port:** 443 / tcp

**Description:** The remote service encrypts traffic using TLS / SSL but allows a client to insecurely renegotiate the connection after the initial handshake. An unauthenticated, remote attacker may be able to leverage this issue to inject an arbitrary amount of plaintext into the beginning of the application protocol stream, which could facilitate man-in-the-middle attacks if the service assumes that the sessions before and after renegotiation are from the same 'client' and merges them at the application layer.

**Solution:** Contact the vendor for specific patch information.

**Business Impact:** If SSL/TLS renegotiation is not properly secured, attackers may exploit this weakness to perform a Man-in-the-Middle (MITM) attack and inject malicious plaintext data into the encrypted communication session. This could allow attackers to manipulate requests sent by users, potentially leading to unauthorized actions being performed on the server. As a result, sensitive information such as authentication data, session tokens, or confidential transactions may be compromised, affecting the integrity and security of the application.

### 2)Vulnerability Name: SSL / TLS Versions Supported

**Severity:** Info

**Plugin:** 56984

**Port:** 443 / tcp

**Description:** This plugin detects which SSL and TLS versions are supported by the remote service for encrypting communications.

**Solution:** Disable outdated and insecure SSL/TLS versions such as SSLv2, SSLv3, and TLS 1.0/1.1 on the server. Configure the server to support only secure protocols like TLS 1.2 or TLS 1.3 with strong cipher suites.

**Business Impact:** If outdated or insecure SSL/TLS versions are supported, attackers may exploit known cryptographic weaknesses to intercept or decrypt secure communications. This can allow unauthorized parties to access sensitive information such as login credentials, personal data, or financial details transmitted between users and the server. As a result, it may lead to data breaches, loss of confidentiality, and potential damage to the organization's reputation.

### 3)Vulnerability Name: SSL/TLS Recommended Cipher Suites

**Severity:** Info

**Plugin:** 156899

**Port:** 443 / tcp

**Description:** The remote host has open SSL/TLS ports which advertise discouraged cipher suites. It is recommended to only enable support for the following cipher suites:

#### **TLSv1.3:**

- 0x13,0x01 TLS13\_AES\_128\_GCM\_SHA256
- 0x13,0x02 TLS13\_AES\_256\_GCM\_SHA384
- 0x13,0x03 TLS13\_CHACHA20\_POLY1305\_SHA256

#### **TLSv1.2:**

- 0xC0,0x2B ECDHE-ECDSA-AES128-GCM-SHA256
- 0xC0,0x2F ECDHE-RSA-AES128-GCM-SHA256
- 0xC0,0x2C ECDHE-ECDSA-AES256-GCM-SHA384
- 0xC0,0x30 ECDHE-RSA-AES256-GCM-SHA384
- 0xCC,0xA9 ECDHE-ECDSA-CHACHA20-POLY1305
- 0xCC,0xA8 ECDHE-RSA-CHACHA20-POLY1305

This is the recommended configuration for the vast majority of services, as it is highly secure and compatible with nearly every client released in the last five (or more) years.

**Solution:** Only enable support for recommended cipher suites.

**Business Impact:** If weak or insecure cipher suites are enabled in SSL/TLS configurations, attackers may exploit cryptographic weaknesses to decrypt or manipulate secure communications. This can lead to exposure of sensitive information such as login credentials, personal data, or financial transactions, potentially resulting in data breaches and loss of user trust.

**4)Vulnerability Name:** SSL/TLS Diffie-Hellman Modulus  $\leq$  1024 Bits (Logjam)

**Severity:** Low

**Plugin:** 83875

**Port:** 443 / tcp

**Description:** The remote host allows SSL/TLS connections with one or more Diffie-Hellman moduli less than or equal to 1024 bits. Through cryptanalysis, a third party may be able to find the shared secret in a short amount of time (depending on modulus size and attacker resources). This may allow an attacker to recover the plaintext or potentially violate the integrity of connections.

**Solution:** Reconfigure the service to use a unique Diffie-Hellman moduli of 2048 bits or greater.

**Business Impact:** If the server uses weak Diffie-Hellman parameters ( $\leq 1024$  bits), attackers may exploit the Logjam vulnerability to downgrade secure connections and decrypt encrypted communication. This could allow unauthorized access to sensitive information such as login credentials, session data, or confidential transactions, compromising the confidentiality of the system.

**5)Vulnerability Name:** SSL Version 2 and 3 Protocol Detection

**Severity:** Critical

**Plugin:** 20007

**Port:** 443 / tcp

**Description:** The remote service accepts connections encrypted using SSL 2.0 and/or SSL 3.0. These versions of SSL are affected by several cryptographic flaws, including:

- An insecure padding scheme with CBC ciphers.
- Insecure session renegotiation and resumption schemes.

An attacker can exploit these flaws to conduct man-in-the-middle attacks or to decrypt communications between the affected service and clients.

Although SSL/TLS has a secure means for choosing the highest supported version of the protocol (so that these versions will be used only if the client or server support nothing better), many web browsers implement this in an unsafe way that allows an attacker to downgrade a connection (such as in POODLE). Therefore, it is recommended that these protocols be disabled entirely.

NIST has determined that SSL 3.0 is no longer acceptable for secure communications. As of the date of enforcement found in PCI DSS v3.1, any version of SSL will not meet the PCI SSC's definition of 'strong cryptography'.

**Solution:** Consult the application's documentation to disable SSL 2.0 and 3.0. Use TLS 1.2 (with approved cipher suites) or higher instead.

**Business Impact:** If SSLv2 or SSLv3 protocols are enabled on the server, attackers can exploit known vulnerabilities in these outdated protocols to decrypt secure communications. This may expose sensitive information such as login credentials, personal data, or session information, leading to potential data breaches and security risks.

**6)Vulnerability Name:** Nessus SYN scanner

**Severity:** Info

**Plugin:** 11219

**Port:** 8080 / tcp / www

**Description:** This plugin is a SYN 'half-open' port scanner. It shall be reasonably quick even against firewalled target, that SYN scans are less intrusive than TCP (full connect) scans against broken services, but they might cause problems for less robust firewalls and also leave unclosed connections on the remote target, if the network is loaded.

**Solution:** Protect your target with an IP filter.

**Business Impact:** If open ports are detected through SYN scanning, attackers can identify active services running on the system and use this information to target potential vulnerabilities. This may increase the risk of unauthorized access, exploitation of services, and potential compromise of the system.

**7)Vulnerability Name:** JQuery 1.2 < 3.5.0 Multiple XSS

**Severity:** Medium

**Plugin:** 136929

**Port:** 8080 / tcp / www

**Description:** According to the self-reported version in the script, the version of JQuery hosted on the remote web server is greater than or equal to 1.2 and prior to 3.5.0. It is, therefore, affected by multiple cross site scripting vulnerabilities, the vulnerabilities referenced in this plugin have no security impact on PAN-OS, and/or the scenarios required for successful exploitation do not exist on devices running a PAN-OS release.

**Solution:** Upgrade to JQuery version 3.5.0 or later.

**Business Impact:** Using vulnerable versions of jQuery may allow attackers to exploit Cross-Site Scripting (XSS) vulnerabilities by injecting malicious scripts into the web application. This can lead to theft of user session cookies, unauthorized access to user accounts, or manipulation of website content, compromising user data and application security.

**8)Vulnerability Name:** Integrate Zscaler with Identity Providers (IdPs) such as Azure AD

**Severity:** Info

**Plugin:** 19506

**Description:** This plugin displays, for each tested host, information about the scan itself :

- The version of the plugin set.
- The type of scanner (Nessus or Nessus Home).
- The version of the Nessus Engine.
- The port scanner(s) used.

- The port range scanned.
- The ping round trip time
- Whether credentialed or third-party patch management checks are possible.
- Whether the display of superseded patches is enabled
- The date of the scan.
- The duration of the scan.
- The number of hosts scanned in parallel.
- The number of checks done in parallel.

**Solution:** Restrict access to scan reports and configuration details to authorized administrators only.

**Business Impact:** If scan configuration details are exposed, attackers may gain information about the security scanning process, which could help them understand the system environment and plan targeted attacks.

## 9)Vulnerability Name: Shor's Harvest Now Decrypt Later

**Severity:** Info

**Plugin:** 298387

**Description:** This plugin reports network services that may be vulnerable now to a future attack by adversaries using a cryptographically relevant quantum computer (CRQC). Shor's is a theoretical algorithm that leverages the unique ability of quantum computation to do massively parallel calculations developed by Peter Shor in 1994.

This algorithm easily computes two classically difficult mathematical problems used in modern cryptography; discrete logarithms, and factoring numbers formed by multiplying large primes. Shor's reduces both of these problems from taking exponential time in chosen cases to being solvable in polynomial time.

Asymmetric encryption algorithms such as RSA, Diffie-Hellman and Elliptic Curve Diffie-Hellman are impacted by Shor's Algorithm. The most common uses of these algorithms are in symmetric key establishment and authentication. These uses render Shor's Algorithm particularly dangerous because it may give an adversary the ability to harvest network

communications now, and in the future, when a CRQC becomes available, extract the symmetric key and decrypt the communication.

**Solution:** Replace affected ciphers with algorithms chosen to resist CRQC attack.

**Business Impact:** If weak cipher algorithms are not replaced, attackers may exploit cryptographic weaknesses or future CRQC (quantum) attacks to decrypt sensitive communications. This could expose confidential data such as login credentials and private information, leading to data breaches, financial loss, and damage to the organization's reputation.

## 10)Vulnerability Name: Patch Report

**Severity:** Info

**Plugin:** 66334

**Description:** The remote host is missing one or more security patches. This plugin lists the newest version of each patch to install to make sure the remote host is up-to-date. Because the 'Show missing patches that have been superseded' setting in your scan policy depends on this plugin, it will always run and cannot be disabled.

**Solution:** Install the patches listed below.

```
. You need to take the following 3 actions :

[Apache Tomcat 7.0.0 < 7.0.108 multiple vulnerabilities (147163)]
+ Action to take : Upgrade to Apache Tomcat version 7.0.108 or later.
+Impact : Taking this action will resolve 32 different vulnerabilities (CVEs)
more...
```

To see debug logs, please visit individual host

**Business Impact:** If missing patches are identified and not applied, attackers may exploit known vulnerabilities in the system. This could lead to unauthorized access, data breaches, system compromise, or service disruption, affecting business operations and security.



## STAGE – 3

**Title:** Integrate Zscaler with Identity Providers (IdPs) such as Azure AD

### • Overview of Zscaler and Identity Provider Integration

Integrating Zscaler with Identity Providers (IdPs) such as Microsoft Azure Active Directory enables organizations to implement a secure and centralized authentication framework. Instead of relying on traditional network-based security models, this integration shifts security control to identity-based access. Users authenticate through the IdP, and Zscaler enforces access policies based on verified identity. This approach simplifies access management while improving overall security posture. By combining Zscaler's cloud-delivered security platform with Azure AD's identity management capabilities, organizations can provide secure access to applications from any location while maintaining strict authentication controls.

### • Role of Azure Active Directory in Identity Management

Azure Active Directory acts as the central identity provider responsible for authenticating users and managing their access privileges. It stores user identities, group memberships, and authentication policies within a cloud-based directory service. When integrated with Zscaler, Azure AD validates user credentials and provides authentication tokens that allow secure access to resources. This centralized identity management eliminates the need for multiple login systems and enables features such as Multi-Factor Authentication (MFA), conditional access policies, and identity governance. As a result, organizations gain improved visibility and control over user access activities.

### • Single Sign-On (SSO) Implementation with Zscaler

Single Sign-On is a key benefit of integrating Zscaler with Azure AD. SSO allows users to authenticate once using their enterprise credentials and gain access to multiple applications without repeated logins. When a user attempts to access a protected application, Zscaler redirects the authentication request to Azure AD. Once the user is successfully verified, Azure

AD sends a secure authentication response back to Zscaler, allowing access to the requested resource. This process improves user experience by reducing login complexity while maintaining strong authentication security.

- **Secure Authentication Using SAML Protocol**

The integration between Zscaler and Azure AD typically uses the Security Assertion Markup Language (SAML) protocol for secure authentication. SAML enables the exchange of authentication information between the identity provider and the service provider. When a user attempts to access Zscaler services, Azure AD generates a SAML assertion containing verified identity information. Zscaler validates this assertion and grants access based on configured policies. This protocol ensures that sensitive authentication information is transmitted securely and prevents unauthorized access attempts.

- **Automated User Provisioning and Lifecycle Management**

User provisioning is an important component of identity integration. By using automated provisioning methods such as SCIM (System for Cross-domain Identity Management), user accounts and group memberships can be synchronized between Azure AD and Zscaler. When new users are added, modified, or removed in Azure AD, these changes are automatically reflected in Zscaler. This automation reduces administrative workload, eliminates manual account management, and ensures that user access permissions remain accurate and up to date across the organization.

- **Security Benefits of Identity-Based Access Control**

Integrating Zscaler with Azure AD strengthens security by implementing identity-based access control. Access decisions are made based on user identity, device status, location, and security policies rather than network location alone. This approach supports Zero Trust principles where every access request is continuously verified. It also helps prevent unauthorized access, reduces the risk of lateral movement within the network, and ensures that only authorized users can access specific applications. As a result, organizations achieve stronger protection against modern cyber threats while maintaining flexible and secure remote access.

## **Conclusion:**

This project focuses on improving security in modern cloud environments by combining vulnerability assessment with identity-based access control. By analyzing system vulnerabilities and implementing secure authentication methods, organizations can protect their applications and data from cyber threats. The integration of security platforms and identity providers helps ensure that only authorized users can access organizational resources while maintaining secure and efficient access management.

### **Stage 1**

In conclusion, Stage 1 focused on understanding the concept of integrating Zscaler with Identity Providers such as Microsoft Azure Active Directory. This stage explained how modern organizations require identity-based access control instead of traditional perimeter-based security models. The integration enables secure authentication mechanisms such as Single Sign-On and Multi-Factor Authentication, ensuring that only verified users can access organizational resources. By combining the cloud security capabilities of Zscaler with centralized identity management, organizations can enforce strong security policies while providing seamless access to applications.

### **Stage 2**

Stage 2 concentrated on vulnerability assessment using the tool Nessus developed by Tenable, Inc.. Through vulnerability scanning of the target system, various security issues related to SSL/TLS configurations, outdated protocols, and application vulnerabilities were identified. The analysis of these vulnerabilities helped in understanding potential risks that could affect system security. By studying the scan reports and remediation suggestions, this stage demonstrated how vulnerability assessment tools help organizations detect weaknesses and strengthen their cybersecurity defenses.

### **Stage 3**

Stage 3 focused on implementing identity-based security through the integration of Zscaler with Azure Active Directory. This stage explained how authentication protocols such as SAML and identity provisioning mechanisms enable secure communication between Zscaler and the identity provider. The integration supports secure access control, automated user provisioning, and role-based access management. By adopting a Zero Trust security approach, organizations can ensure that every access request is verified before granting application access. This stage highlighted how identity-centric security architecture improves both security and user experience in cloud-based environments.

Overall, the project demonstrated how modern cybersecurity strategies combine identity management, vulnerability assessment, and cloud-based security platforms to protect organizational resources. By integrating Zscaler with Azure Active Directory and analyzing system vulnerabilities through Nessus scanning, organizations can establish a secure and scalable security infrastructure for cloud environments.

## **Future Scope:**

### **1) Future of Web Application Security Testing**

The future of web application security testing will become more advanced as web applications continue to grow in complexity and scale. Organizations depend heavily on web applications to manage data, provide services, and support business operations. Therefore, ensuring the security and reliability of these applications will remain a major priority.

**Some important trends that will influence the future of web application security testing include:**

- **Greater focus on security testing:** As cyber attacks continue to increase, organizations will place greater emphasis on identifying vulnerabilities such as injection attacks, cross-site scripting (XSS), and authentication weaknesses. Security testing tools will become more advanced to detect these threats earlier.
- **Use of Artificial Intelligence (AI) and Machine Learning (ML):** AI and ML technologies will help automate vulnerability detection and identify complex attack patterns. These technologies can analyze large amounts of security data and detect anomalies that traditional testing methods may miss.
- **Cloud-based testing environments:** With more applications being deployed in the cloud, testing platforms will increasingly move to cloud environments. Cloud-based security testing provides scalability, flexibility, and easier integration with modern application infrastructures.
- **Improved user experience and security testing:** Future testing will focus not only on security vulnerabilities but also on ensuring that secure systems provide a smooth and efficient user experience.

### **2) Future Scope of Vulnerability Assessment and Testing Process**

The vulnerability assessment and testing process will continue to evolve as organizations adopt more complex digital systems and cloud infrastructures. Security testing will play a critical role in identifying potential threats before they can be exploited.

### **Key future developments include:**

- **Continuous vulnerability monitoring:** Instead of performing occasional scans, organizations will adopt continuous scanning and monitoring to detect vulnerabilities in real time.
- **Automation in security testing:** Automated vulnerability scanners and testing tools will help reduce manual effort and improve the speed of detecting security issues.
- **Integration with DevSecOps:** Security testing will be integrated directly into the software development lifecycle (SDLC) so that vulnerabilities can be detected and fixed during development.
- **Advanced testing tools:** Future security tools will include intelligent scanning capabilities, automated remediation suggestions, and deeper analysis of system configurations.

### **3) Future Scope of ZTNA, SOC, and SIEM**

The future of Zero Trust Network Access (ZTNA), Security Operations Centers (SOC), and Security Information and Event Management (SIEM) systems will focus on improving real-time threat detection and automated security management.

### **Important future developments include:**

- **AI-based threat detection:** Security platforms will use artificial intelligence to detect suspicious behavior, identify security threats, and respond automatically to potential attacks.
- **Identity-based security frameworks:** Identity will become the central element of security systems. Technologies such as multi-factor authentication, behavioral biometrics, and identity analytics will improve authentication accuracy.
- **Integration with cloud security platforms:** Security solutions such as Zscaler will increasingly integrate with cloud identity providers like Azure Active Directory to provide centralized access control and secure application access.
- **Automation and orchestration:** Future SOC and SIEM platforms will automate threat detection, incident response, and security monitoring to reduce response time and improve efficiency.

- **Improved threat intelligence sharing:** Security systems will collect and analyze threat intelligence from multiple sources, helping organizations detect new and emerging cyber threats more effectively.

### **Topics Explored:**

Identity and Access Management (IAM), Cloud Security Architecture, Zero Trust Network Access (ZTNA), Single Sign-On (SSO), Multi-Factor Authentication (MFA), Security Assertion Markup Language (SAML) Authentication, User Provisioning and Identity Synchronization, Vulnerability Assessment and Security Testing, SSL/TLS Security Configuration, Web Application Security (XSS and Injection), Security Monitoring and Logging, Integration of Cloud Security Platforms with Identity Providers.

### **Tools Explored:**

Zscaler Cloud Security Platform, Microsoft Azure Active Directory (Azure AD), Nessus Vulnerability Scanner, Web Application Security Testing Platform “[zero.webappsecurity.com](https://www.zero.webappsecurity.com)”, Web Browser Developer Tools, SSL/TLS Configuration Analysis Tools.

THE END